# Introduction aux tests unitaires

Michaël Soulignac

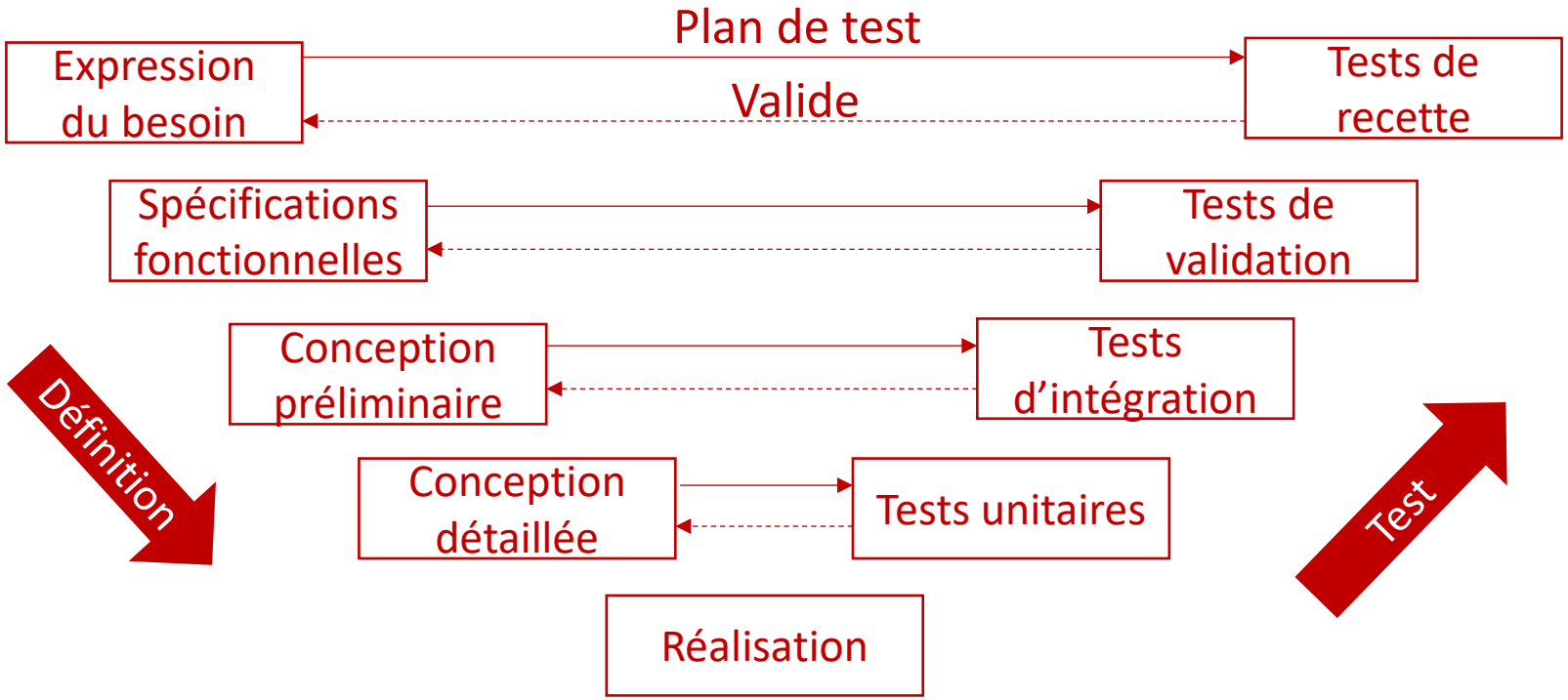
ESILV Nantes

# Concept de test unitaire

# Pourquoi tester ?

- S'assurer que le code produit répond au besoin initial

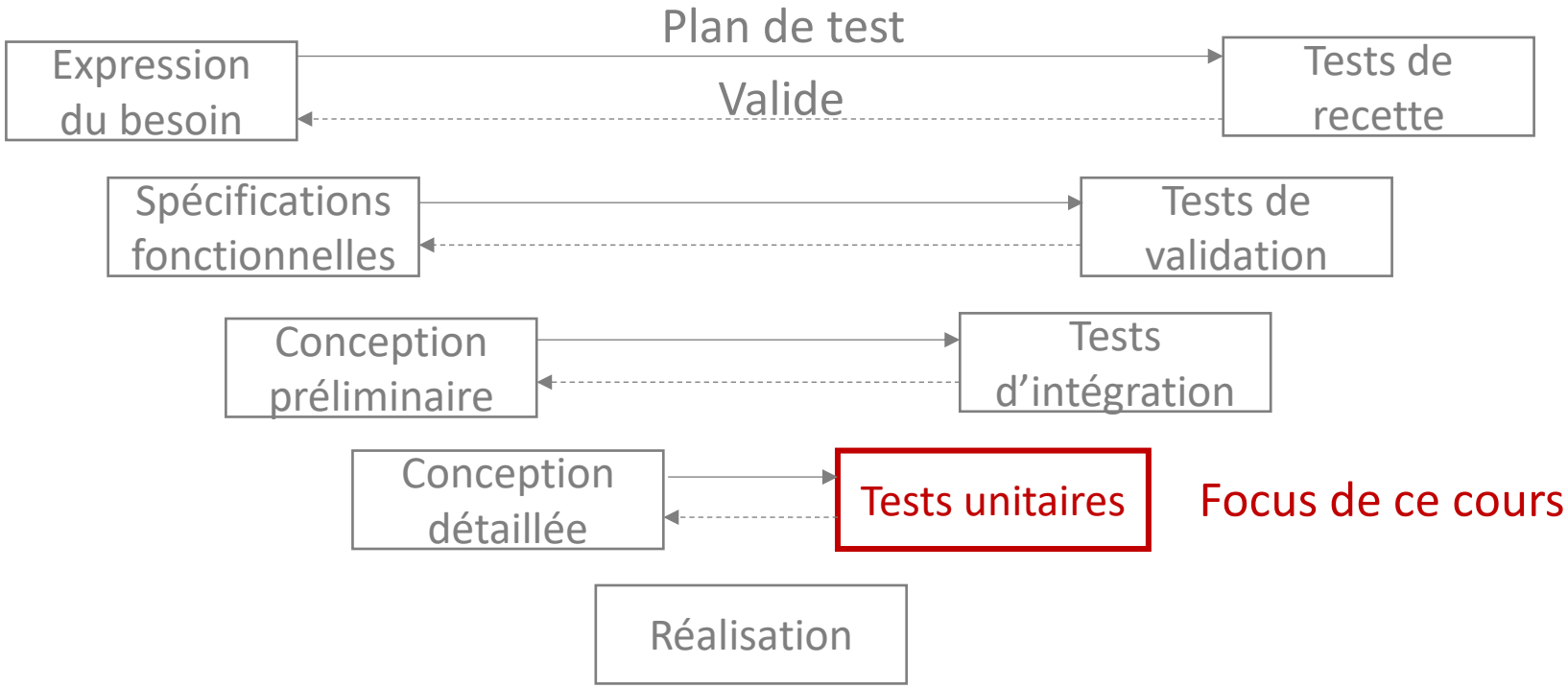
⇒ Exemple : cycle en V



# Pourquoi tester ?

- S'assurer que le code produit répond au besoin initial

⇒ Exemple : cycle en V



# Pourquoi tester ?

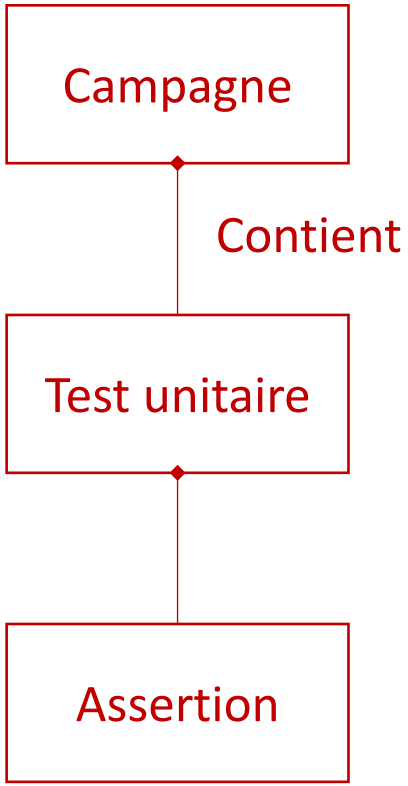
- S’assurer que le code produit répond au besoin initial
- S’assurer qu’une évolution du code (ajout de fonctionnalité, correction de bug…) n’introduit pas de nouveaux bugs dans les fonctionnalités existantes

⇒ Tests de non-régression

# Ecriture des tests unitaires

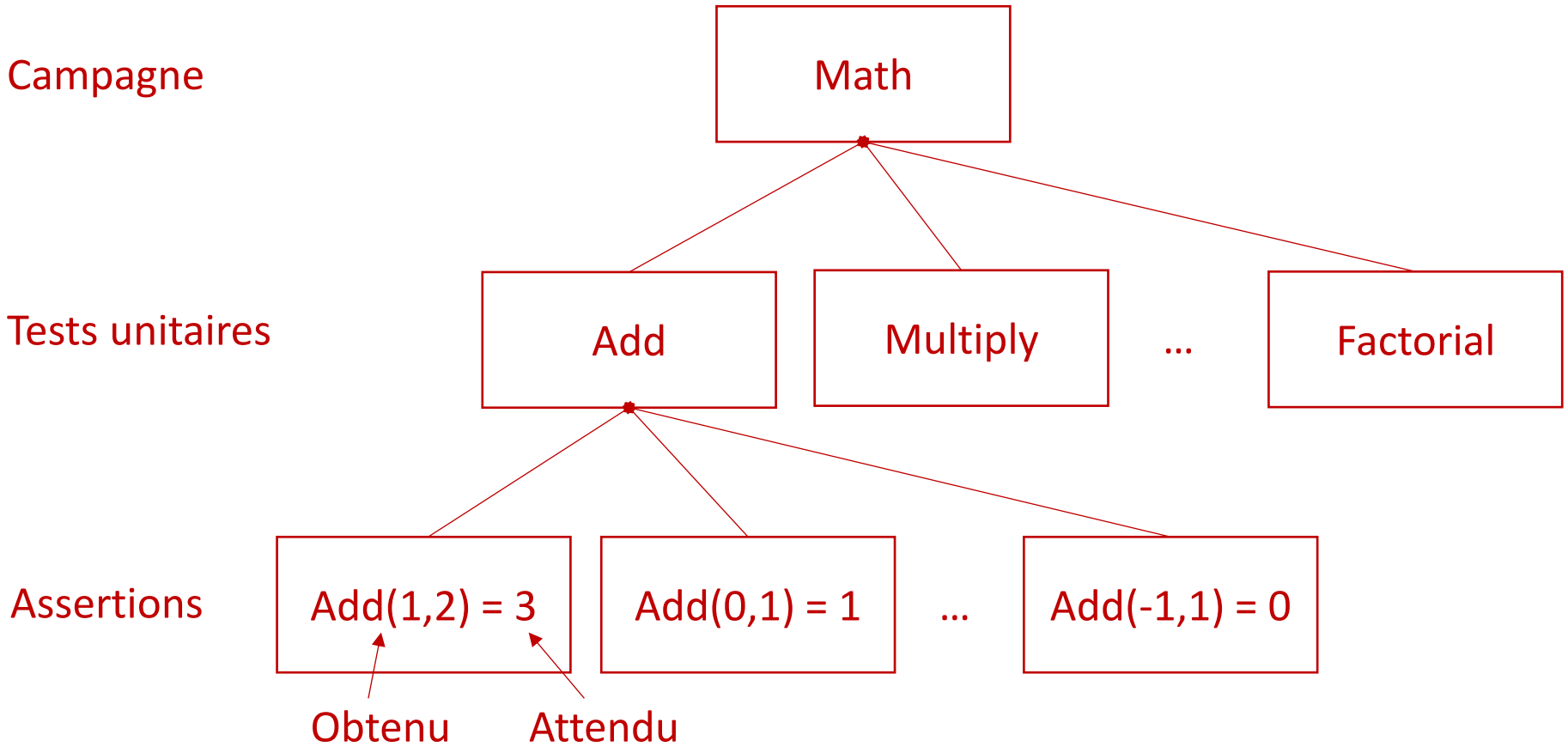
- Avant la réalisation
- Idéalement, par des personnes différentes des personnes en charge de la réalisation

# Structure des tests unitaires




Résultat attendu = résultat obtenu ?

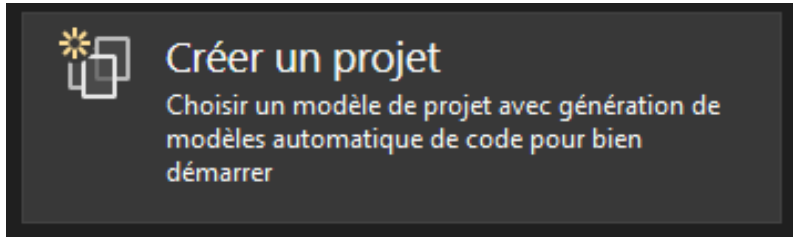
# Structure des tests unitaires

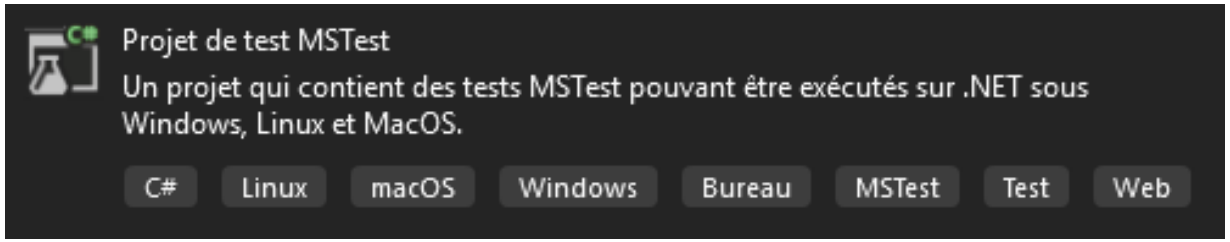


# Tests unitaires dans Visual Studio

# Projet MSTest

- Visual Studio intègre un *framework* de test appelé MSTest
- Pour l'utiliser, il suffit de créer un projet de type MSTest

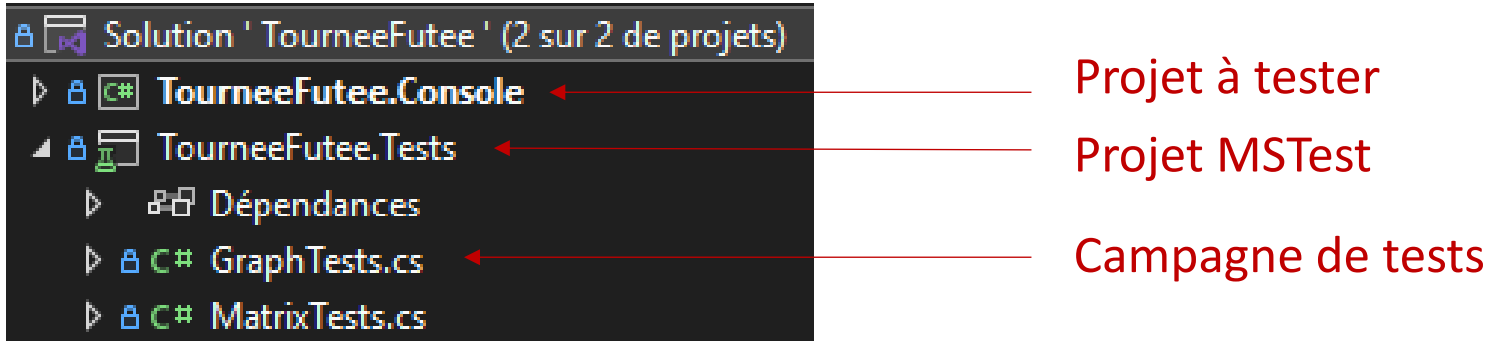


# Intégration dans une solution Visual Studio

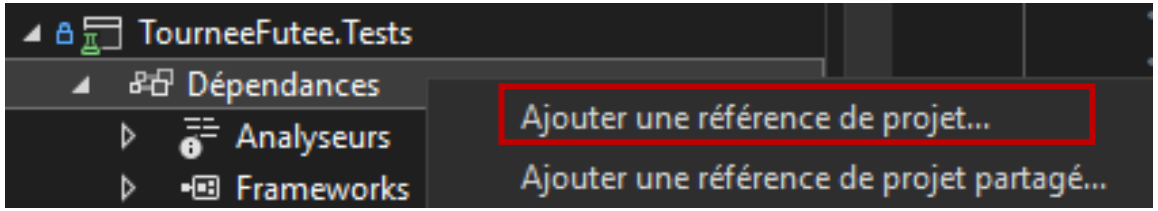
Une solution intégrant des tests inclut deux projets :

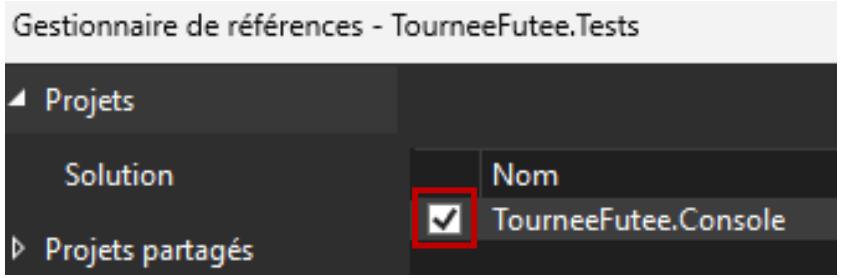
- Le projet à tester (la réalisation). Exemple : projet console.
- Le projet MSTest contenant les campagnes de tests
- Exemple :

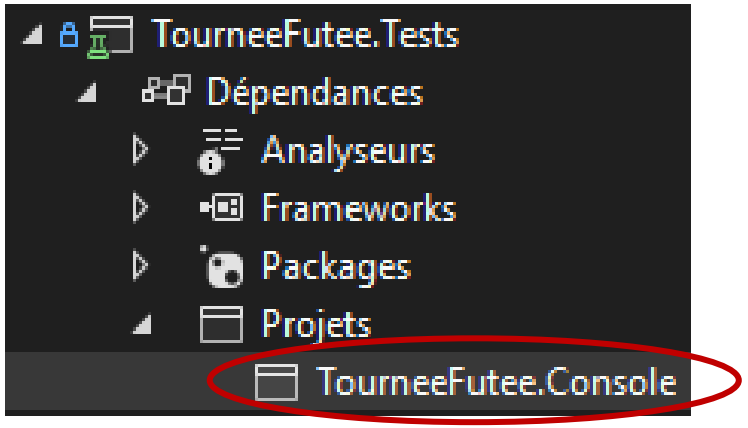


# Intégration dans une solution Visual Studio

Le projet MSTest doit référencer explicitement le projet à tester pour pouvoir utiliser son code :

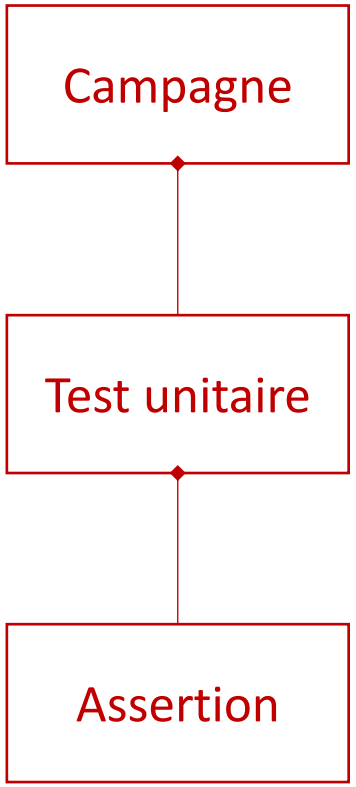






# Tests unitaires dans Visual Studio

# Structure des tests unitaires dans Visual Studio




Classe annotée avec `[TestClass]`

Méthode annotée avec `[TestMethod]`

Une des méthodes de la classe `Assert` ou variantes décrites dans la documentation

# Structure des tests unitaires dans Visual Studio

```
[TestClass]
0 références
public class MathTests
{

    [TestMethod]
    0 références
    public void Add()
    {
        Assert.AreEqual(3, Math.Add(1, 2));
        Assert.AreEqual(1, Math.Add(0, 1));
        Assert.AreEqual(0, Math.Add(-1, 1));
    }
}
```

Campagne

Test unitaire

Assertion

Attendu

Obtenu

# Exécution des tests

- On peut exécuter les tests au niveau de granularité que l'on souhaite :
  - Toutes les campagnes
  - Une campagne en particulier
  - Ou même un seul test unitaire

- Pour cela : Clic droit sur l'élément à tester >

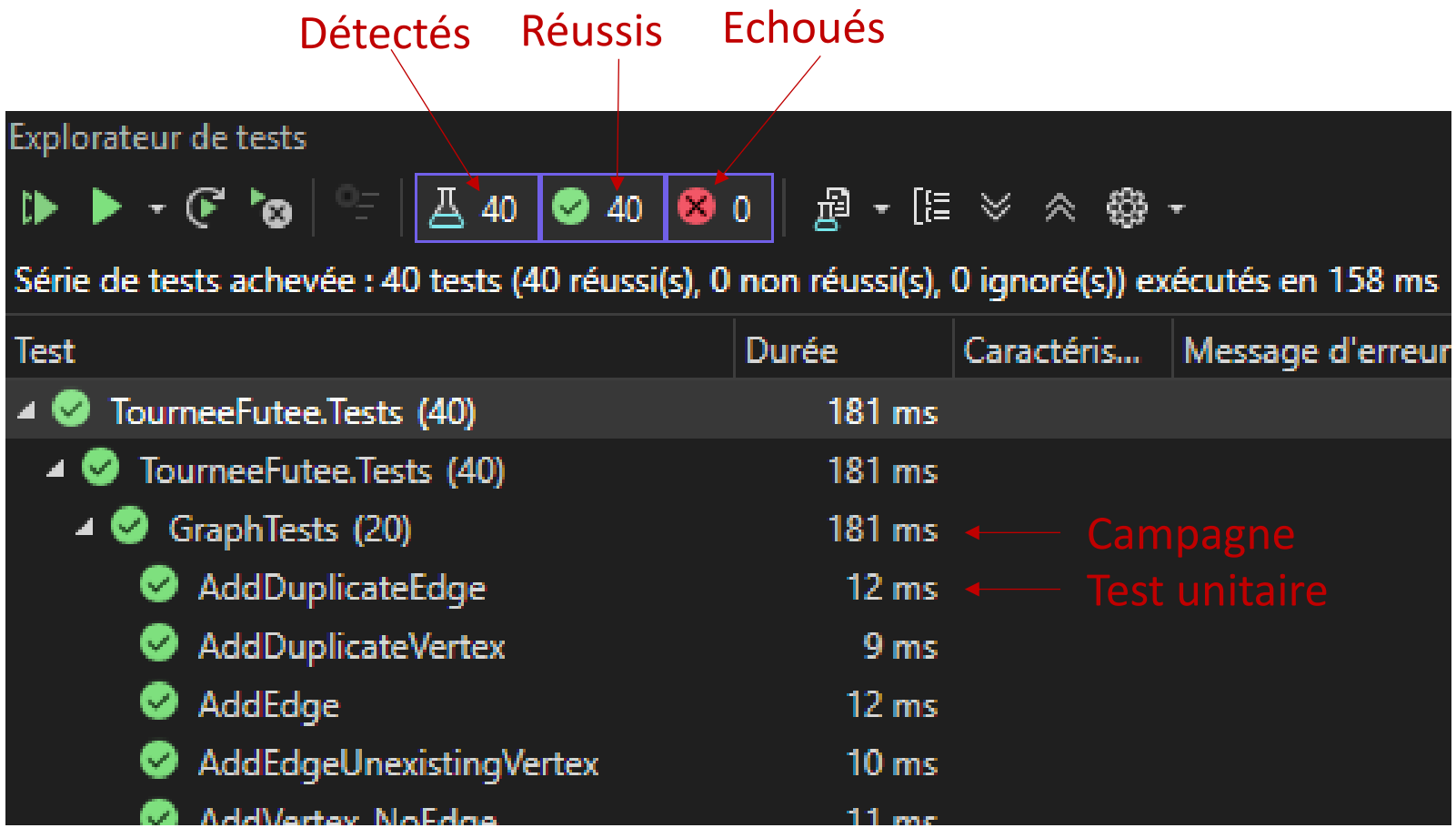
- Les tests sont exécutés en parallèle

# L'explorateur de tests

Fenêtre séparée qui :

- Affiche les statistiques sur les tests
- Liste chaque test avec son statut
- Permet de consulter le détail d'exécution d'un test précis
- (Ré)exécuter tous les tests, une campagne, ou un test spécifique
- Lancer un/des test(s) en mode débogage

# L'explorateur de tests



# Exemple de sortie détaillée (échec)

**Récapitulatif des détails du test**

EmptyGraph

Source : GraphTests.cs ligne 12

Durée : 46 ms

Message :

Échec de Assert.AreEqual. Attendu : <0>, Réel : <-1>.

Arborescence des appels de procédure :

GraphTests.EmptyGraph(Boolean directed) ligne 196

GraphTests.EmptyGraph() ligne 14